BÖLÜM 7: GÜVENLIK VE SIFIR GÜVEN — KRALLIĞI SAVUNMAK

⏱ 9 dk • DERS 26

Parça 26: Ölçekte Kimlik ve Yetkilendirme: OAuth2, OIDC ve RBAC vs. ABAC

Başlık boyutlarını patlatmadan kimlik doğrulama ile ayrıntılı izinleri birbirinden ayırmak.

> 🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**
>
> Bir sağlık teknolojisi platformunda mimarlık ekibi her istekte veritabanına gitmemek için doktorların tüm izinlerini doğrudan JSON Web Token (JWT) içine kodlamaya karar verdi. Büyük hastane zincirlerine bağlı uzman doktorlar için bu izinler; 400 klinik kodu, 80 hastane kanadı ve 50 özel prosedür yetkisi içeriyordu. Altıncı ayda JWT boyutu 28 kilobayta fırladı! Doktorlar acil hasta konsültasyonu açmaya çalıştığında NGINX tersine vekil sunucuları `HTTP 431 Request Header Fields Too Large` hatası vererek doktorları kilitledi. Daha da kötüsü, bir doktorun yetkisi suiistimal yüzünden iptal edildiğinde, stateless JWT 24 saat geçerli kaldığı için doktor sisteme girmeye devam etti!

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Kimlik Doğrulama (AuthN: **Sen kimsin?**) ile Yetkilendirme (AuthZ: **Neleri yapmaya iznin var?**) iki farklı mimari problemdir. OAuth 2.0 ve OIDC kimlik tespitini çözmüştür; ancak ölçeklenebilir yetkilendirme, izinleri devasa JWT token'larına tıkmakla çözülemez. Modern mimariler hafif kimlik token'ları ile harici çalışan dinamik yetkilendirme motorlarını (**RBAC** veya **ABAC**) birbirinden ayırır.

Temel Mimari İlkeler ve Zihinsel Modeller

Kimlik Protokollerinin Özeti

- **OpenID Connect (OIDC):** Kimliği doğrular ve kullanıcı profilini içeren bir `id_token` (JWT) üretir.
- **OAuth 2.0:** Yetkilendirme çerçevesi. İstemcinin kullanıcı adına kaynaklara erişmesini sağlayan `access_token` üretir.
- **RBAC (Rol Tabanlı Erişim):** İzinler rollere atanır (Yönetici, Editör). Basittir ama kurumsal kurallar arttıkça 'Rol Patlaması' yaşanır.
- **ABAC / ReBAC (Özellik Tabanlı Erişim):** Kararlar kurallarla verilir: `Kullanıcı X Belge Y'yi düzenleyebilir Mİ EĞER Kullanıcı.Bölüm == Belge.Bölüm VE Saat \< 18:00?` (Örn: Open Policy Agent - OPA, Oso).

Anında İptal (Revocation) Stratejisi

JWT yaşam sürelerini kısa tutun (5 - 15 dakika). Anında oturum kapatma ve engelleme için Redis üzerinde hafif bir kara liste (blacklist) kontrolü yapın.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **50 KB'lık JWT Canavarı:** Kullanıcının adresini, tüm rollerini ve profil detaylarını HTTP başlığına tıkmak.
- **Kod İçi Sabit İzin Kontrolleri:** Servis kodlarının içine `if (user.rol == 'mudur' && user.ulke == 'TR')` yazıp iş kurallarını 50 yere saçmak.
- **Kendi Kriptografisini Yazmak:** Kendi şifre özetleme veya oturum algoritmasını icat etmeye kalkışmak (Bcrypt, Argon2 varken).

🎯 MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR

- **1. Adım:** OIDC/OAuth 2.0 için hazır bir kimlik sağlayıcı (Auth0, Keycloak veya AWS Cognito) entegre et.
- **2. Adım:** Yetkilendirme kurallarını Open Policy Agent (OPA) gibi harici bir kural motorunda topla.
- **3. Adım:** Access Token süresini 15 dakika yap ve dönen yenileme token'ları (refresh tokens) kullan.

Mimari Çıkarımlar ve Hile Tablosu

- AuthN kapıyı çalanın kim olduğunu söyler; AuthZ hangi odalara girebileceğini belirler.
- Stateless token'ları hafif tutun; ayrıntılı izinleri dinamik yetki motorlarına bırakın.
- Asla kendi güvenlik ve şifreleme algoritmanızı yazmayın; denetlenmiş standartları kullanın.

🎙️ UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU

Ders Değerlendirmesi ve Ödev: Sisteminiz anında yetki iptalini nasıl yönetiyor? Bir çalışanın işine şu an son verilse, açık oturumu hemen kapanır mı yoksa token süresi bitene kadar sisteme erişebilir mi?